

The `remove()` method copies the last item in the heap—the one at `_nItems - 1`—to the root node. It decrements `_nItems` first because the last item in the array was just moved to index 0. After clearing the last cell for the garbage collector, it calls `_siftDown()` to move the new root item down the heap to where it belongs. When the sift down finishes, it can return the item removed from the root node earlier.

The `_siftDown()` method needs to know which nodes are leaf nodes. Because the tree is complete, this index can be calculated from the number of items in the heap. The `firstleaf` index always occurs at index `_nItems // 2`. You can see examples of these index relationships in [Figure 13-4](#). The first tree has five items, and the first leaf is at index 2. The second tree (the leftmost one in the second row) has six items, and its first leaf is at index 3. All nodes with an index equal to or larger than `firstleaf` are leaf nodes. If index `i` for the item to be sifted down is in that range, the item cannot be moved down because it has no child nodes, so the `_siftDown()` method returns immediately.

Listing 13-5 *The `remove()` Method for Heaps*

```
class Heap(object):
...
    def remove(self):          # Remove top item of heap and return it
        if self.isEmpty():    # It's an error if the heap is empty
            raise Exception("Heap underflow")
        root = self._arr[0]    # Store the top item
        self._nItems -= 1      # Decrease item count
        self._arr[0] = self._arr[self._nItems] # Move last to root
        self._arr[self._nItems] = None # Clear for garbage collection
        self._siftDown(0)      # Move last item down into position
        return root            # Return top item

    def _siftDown(self, i):    # Sift item i down to preserve heap cond.
        firstleaf = len(self) // 2 # Get index of first leaf
        if i >= firstleaf:     # If item i is at or below leaf level,
            return             # it cannot be moved down
        item = self._arr[i]    # Store item at cell i
        itemkey = self._key(item) # and its key
        while i < firstleaf:   # While i above leaf level, find children
            left, right = self.leftChild(i), self.rightChild(i)
            maxi = left        # Assume left child has larger key
            if (right < len(self) and # If both children are present, and
                self._key(self._arr[left]) < # left child has smaller
                self._key(self._arr[right])): # key
                maxi = right
            self._arr[i], self._arr[maxi] = self._arr[maxi], self._arr[i]
            i = maxi
```

```

        maxi = right      # then use right child
    if (itemkey <         # If item i's key is less
        self._key(self._arr[maxi])): # than max child's key,
        self._arr[i] = self._arr[maxi] # then move max child up
        i = maxi
    else:                 # If item i's key is greater than or equal
        break             # to larger child, then found position
self._arr[i] = item      # Move item to its final position

```

When `_siftDown()` is called on an internal (nonleaf) node, it stores the `item` at `i` and its key in temporary variables. Then it begins a loop that will descend the heap tree until it reaches a leaf node or finds a node where the stored item can be reinserted to preserve the heap condition.

Inside the loop, it gets the indices of node `i`'s children. If there is only a left child, it must have the maximum key; otherwise, the left and right child keys must be compared to determine the index of the child node with the maximum key, `maxi`. The test starts by assuming `maxi` is the left child. If the right child exists and has a larger key, `maxi` is set to the right child.

Now the key of the item being sifted down can be compared with the maximum child key. If the `itemkey` is smaller than the maximum child key, the maximum child is moved up. That creates a “hole” at the maximum child node, and `i` is updated to point there for the next loop. If the `itemkey` equals or exceeds the maximum child key, index `i` points to where the item belongs, and the loop terminates by using `break`.

The final step of `_siftDown()` moves the `item` to sift down into cell `i`. That fills the last “hole” and restores the heap condition.

Traversal

Traversing the items in a heap is somewhat common. This operation might be used, for example, to list all the processes waiting to run in a priority queue or to visit each item and collect statistics. Because the heap only partially orders the items, it is not easy to traverse the items in the order of their key values. Instead, the simplest traversal order is the same as traversing the underlying array.

[Listing 13-6](#) shows the `traverse()` generator used to step through all the items. The completeness of the heap tree means it can simply step through every

index for active items in the array. That's done with a `for` loop over the range of indices, yielding each array cell content back to the caller.

Listing 13-6 *The `traverse()` Generator and `print()` Method for Heaps*

```
class Heap(object):
...
    def traverse(self):          # Generator to step through all heap items
        for i in range(len(self)): # Get each current item index
            yield self._arr[i] # and yield the item at the index

    def print(
        self, indentBy=2,      # Print heap tree with root on left
        indent='', i=0):      # indenting by a few spaces for each level
        # starting with indent at node i
        if i >= len(self):    # If item i is not in tree
            return            # don't print it
        next = indent + ' ' * indentBy
        self.print(indentBy,  # Print right subtree of i at next indent
                    next, self.rightChild(i))
        print(indent, self._arr[i]) # Print item i after indent, then
        self.print(indentBy,  # Print left subtree of i at next indent
                    next, self.leftChild(i))
```

Although this simple traversal order doesn't follow the key ordering, it does follow a particular order for the heap's binary tree. The order is called **breadth first** because it works across the broad levels of the tree visiting all the nodes at a particular level in the tree before going to the next lower level. The nodes are visited from shallowest to deepest, and the breadth of each level is traversed in order, left to right. Try traversing a medium- or large-sized heap (15 or more items) using the Visualization tool to see this. This ordering is very useful for some tree algorithms.

Printing the heap tree is often useful when developing the code. The `print()` method in Listing 13-6 prints the tree on its "side," as was done with trees in Chapters 8, 9, and 10. The method is recursive and uses a reverse form of in-order traversal, where the right subtree of a node is printed, then the node itself, and then its left subtree. The indentation increases for each subtree.

Efficiency of Heap Operations

For a heap with a substantial number of items, the sift up and sift down algorithms are the most time-consuming part of the operations you've seen.

These algorithms spend time in a loop, repeatedly moving nodes up or down along a path. The maximum number of copies necessary is bounded by the height of the heap; if there are five levels, four copies carry the “hole” from the top to the bottom. (We ignore the two moves used to transfer the end node to and from temporary storage; they’re always necessary, so they require constant time.)

The `_siftUp()` method has up to four operations in its loop: (1) calculate the parent index of the current “hole,” (2) compare the key of the node to insert with that of the parent, (3) copy the parent down to the “hole,” and (4) move the “hole” up. The third and fourth operations are skipped on the final iteration of the loop. The `_siftDown()` method has up to six operations in its loop: (1) calculate the left and right child indices of the current “hole,” (2) compare the keys of the left and right child items, (3) assign the largest child index, (4) compare the largest child key with that of the item sifting down, (5) copy the largest child item into the “hole,” and (6) move the “hole” down. The fifth and sixth operations are skipped on the final iteration of the loop. Both methods must move the sifted node into the final “hole” after the loop exits.

A heap is a special kind of binary tree, and as you saw in [Chapter 8](#), the number of levels L in a binary tree equals $\log_2(N+1)$, where N is the number of nodes.

The `_siftUp()` and `_siftDown()` routines cycle through their loops $L-1$ times, so the first takes time proportional to $\log_2 N$, and the second somewhat more because of the extra comparison and assignment. Thus, the insert and remove operations both operate in $O(\log N)$ time.

For insertion, you must also consider the time to grow the heap array up to the size needed to hold all the heap items. The analysis of the time needed to grow the array follows that of growing the array for hash tables. Remember that doubling the hash array when the load factor was above a threshold allowed a hash table for N items to be built in $O(N)$ time. In the case of heaps, the insert operation is $O(\log N)$ instead of $O(1)$. That means inserting N items into a heap takes $O(N \times \log N)$ time.

Traversing a heap of N items requires $O(N)$ time. There are no empty cells in the array that must be visited, like in the hash table, so traversing a heap is a little faster than a hash table.

A Tree-Based Heap

In the figures of this chapter, we've shown heaps as if they were trees because it's easier to visualize them that way, but the implementation is array-based. It's possible, of course, to use an actual tree-based implementation. The tree will be a binary tree, but it won't be a search tree because, as you've seen, the ordering principle is not as strong. It will be a complete tree, with no missing nodes. Let's call such a tree a *tree-based heap*.

One problem with tree-based heaps is finding the last node. You need to find this node to remove the maximum item because it's the node that's inserted in place of the deleted root (and then sifted down). You also need to find the first "empty node"—the node just after the last node—because that's where the insert method needs to place the new item before sifting it up. In general, you can't search for these nodes because you don't know their values, and anyway it's not a search tree. You could add some fields to a tree-based heap data structure that maintain pointers to the end, just like for double-ended lists, but there's another way that doesn't require the additional fields.

As you saw in the discussion of the Huffman tree in [Chapter 8](#), you can represent the path from root to leaf as a binary number, with the binary digits indicating the path from each parent to its child: 0 for left and 1 for right. It turns out there's a simple relationship between the number of nodes in the tree and the binary number that codes the path to the last node. Assume the root is numbered 1; the next row has nodes 2 and 3; the third row has nodes 4, 5, 6, and 7; and so on. This is, of course, one more than the array index used in the array-based implementation.

To find the last filled node (or the first empty node), convert the number of nodes (or one more than the number) to binary. For example, say there are 29 nodes in the tree and you want to find the last node. The number 29 (decimal) is 11101 in binary. Remove the initial 1, leaving 1101. This is the path from the root to node 29: right, right, left, right. Each 1 in the binary representation means take the right child, and each 0 means the left. The first available null node will be found by following the path to 30, which (after removing the initial 1) is 1110 binary: right, right, right, left. shows the path through the tree.

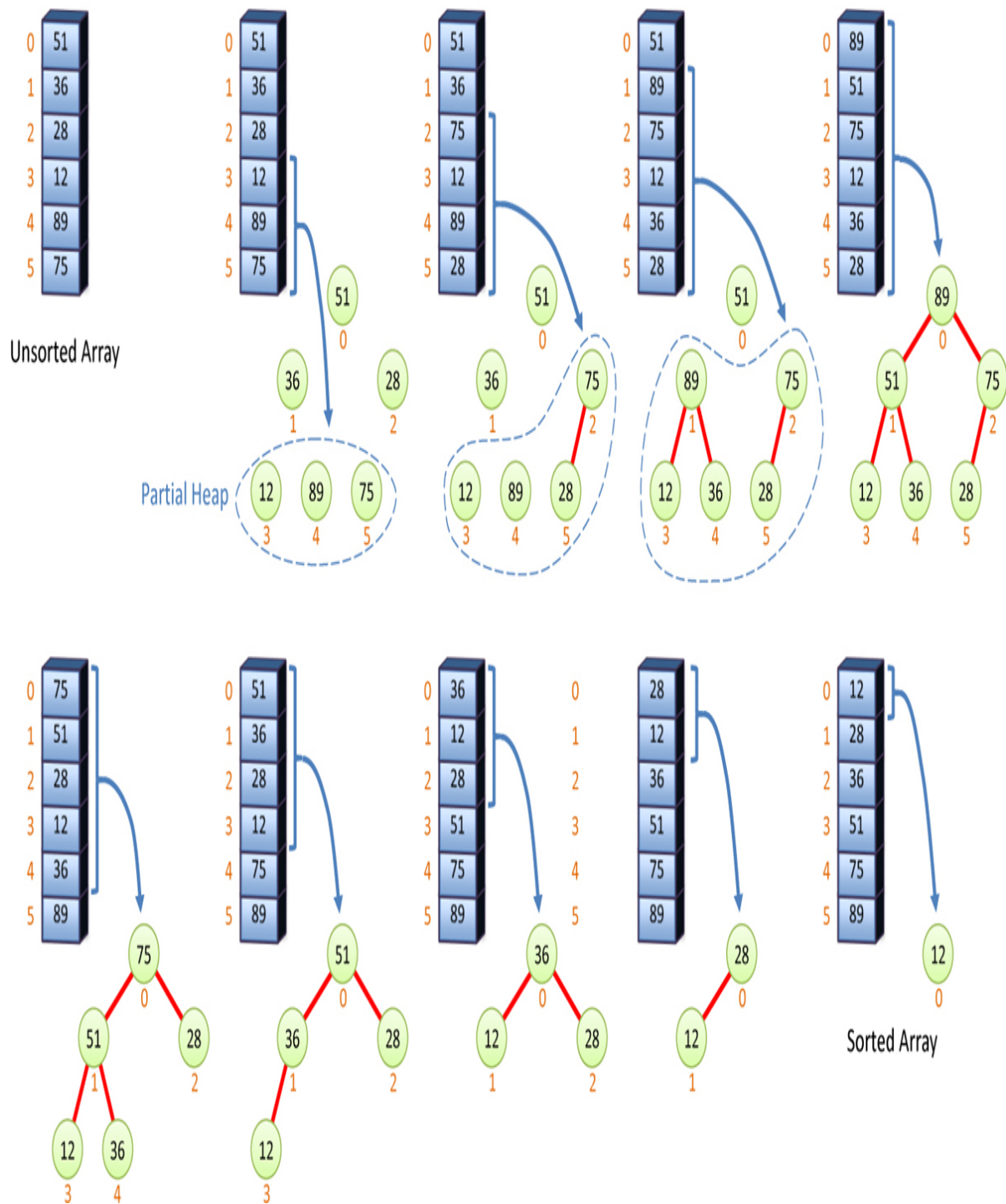


Figure 13-10 *Finding the path to the last node in a 29-node tree.*

You can see the binary representation of an integer in Python using the string format tool. For example, `'{:b}'.format(29)` evaluates to `'11101'`. It's also easy to repeatedly use the `%` operator to find the remainder (0 or 1) when the

number `n` is divided by 2 and then use the `//` or `>>` operators to divide `n` by 2 and drop the last bit. When `n` is less than 1, you're done. The sequence of remainders, which you can save in an array, is the binary number. (Be careful which order you interpret them!)

Using the binary representation allows a tree-based heap to find the path to the last leaf or next empty leaf, but it doesn't speed up reaching that node. The algorithm still must follow all the references from the root to the node, taking $O(\log N)$ steps. For an array-based heap, that can be done in constant time, $O(1)$.

After the appropriate node (or null child) is found, the heap operations are straightforward. When you're sifting up or down, the structure of the tree doesn't change, so you don't need to change linkages to move the actual nodes around. You can simply copy the data items from one node to the next, like was done in the array-based implementation. This way, you don't need to connect and disconnect all the children and parents for a simple move. The `Node` class, however, needs a field for the parent node in addition to the child nodes because you need to access the parent when you sift up. This is somewhat like the doubly linked lists you saw in [Chapter 5](#), "[Linked Lists](#)." The root of the tree should also keep an `nItem` field to facilitate finding the path to the last leaf and first empty node.

In the tree heap insertion and removal operations take $O(\log N)$ time. As in the array-based heap, the time is mostly spent doing the sift up and sift down operations, which take time proportional to the height of the tree. Traversal of a tree-based heap still takes $O(N)$ time, but the traversal order can be any of the orders you saw for binary trees: pre-order, in-order, or post-order. While the name might be "in-order," the traversal is still not in the order of the item keys. The flexibility to perform those three orderings comes from the explicit child references in the structure. It's possible to do a breadth-first traversal by converting node numbers to paths from the root, and then following those paths to the node, but that would take significantly more than $O(N)$ time.

Heapsort

The efficiency of the heap data structure lends itself to a surprisingly simple and very efficient sorting algorithm called a **heapsort**.

The basic idea is to insert all the unordered items from a source data structure into a heap using the normal `insert()` method. Repeated application of the `remove()` method then removes the items in sorted order. Here's how that might look to sort a Python sequence into a result array:

```
theHeap = Heap(size=len(aSequence)) # Create an empty heap
for item in aSequence:               # Loop over unsorted sequence
    theHeap.insert(item)              # Copy items to heap
result = []                          # Make a result array
while not theHeap.isEmpty():          # Loop over array indices
    result.append(theHeap.remove())    # Copy items back to array
```

This code would put the items in the result array in descending order because the first item removed from the heap is the maximum. To put the items in ascending order, you could construct the result array to have enough cells and then index it in reverse order.

Because `insert()` and `remove()` operate in $O(\log N)$ time, and each must be applied N times, the entire sort requires $O(N \times \log N)$ time, which is the same as the quicksort. It's not quite as fast as quicksort overall, however, partly because there are more operations in the inner `while` loop in `_siftDown()` than in the inner loop in quicksort.

With a little cleverness, we can enhance this basic algorithm to make heapsort more efficient. The first enhancement saves time, and the second saves memory.

Sifting Down Instead of Up

If you insert N new items into a heap, you apply the `_siftUp()` method N times. You can take advantage, however, of a similar technique to what you did when removing items: sifting an out-of-sequence item down from the root into a correctly arranged heap. What's more, doing this only needs $N/2$ calls to `_siftDown()`, because you need to do it only for internal (nonleaf) nodes. This approach offers a small speed advantage, even though sifting down is slightly more time-consuming.

Two Correct Subheaps Make a Correct Heap

To see how this approach works, we need to look at smaller parts of the heaps: **subheaps**. What do we mean by subheap? They are just like subtrees inside of

trees. Any node in a heap tree can be thought of as the root of a subheap. [Figure 13-11](#) shows three subheaps outlined with dotted triangles.

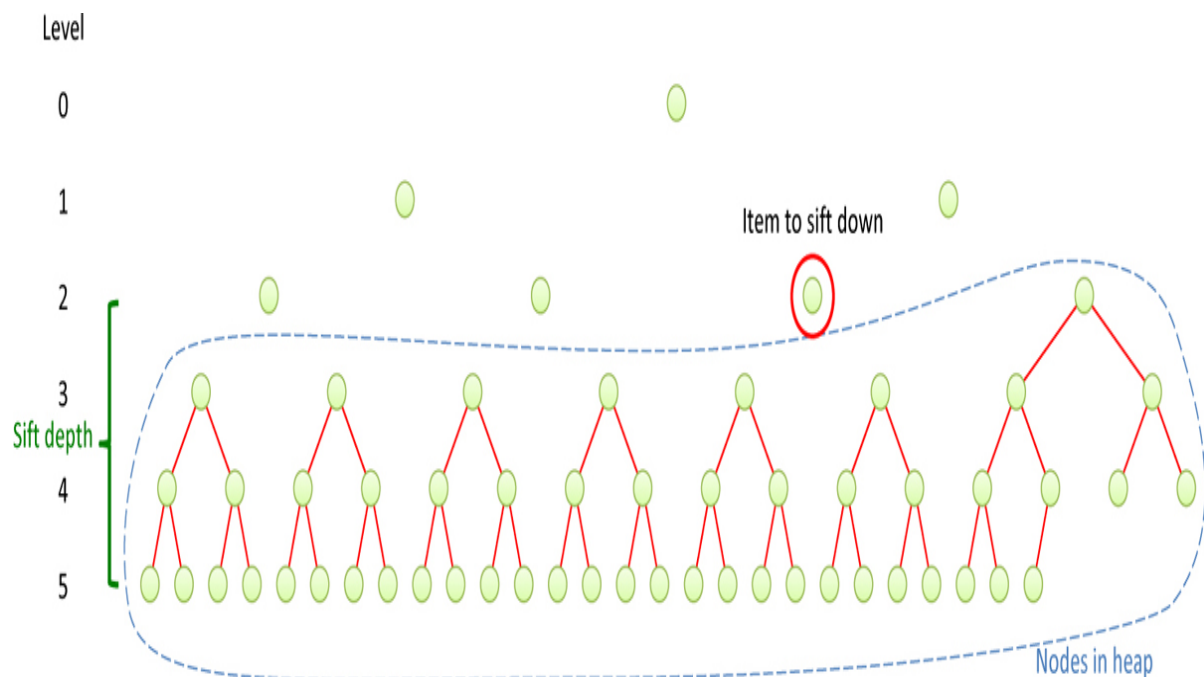


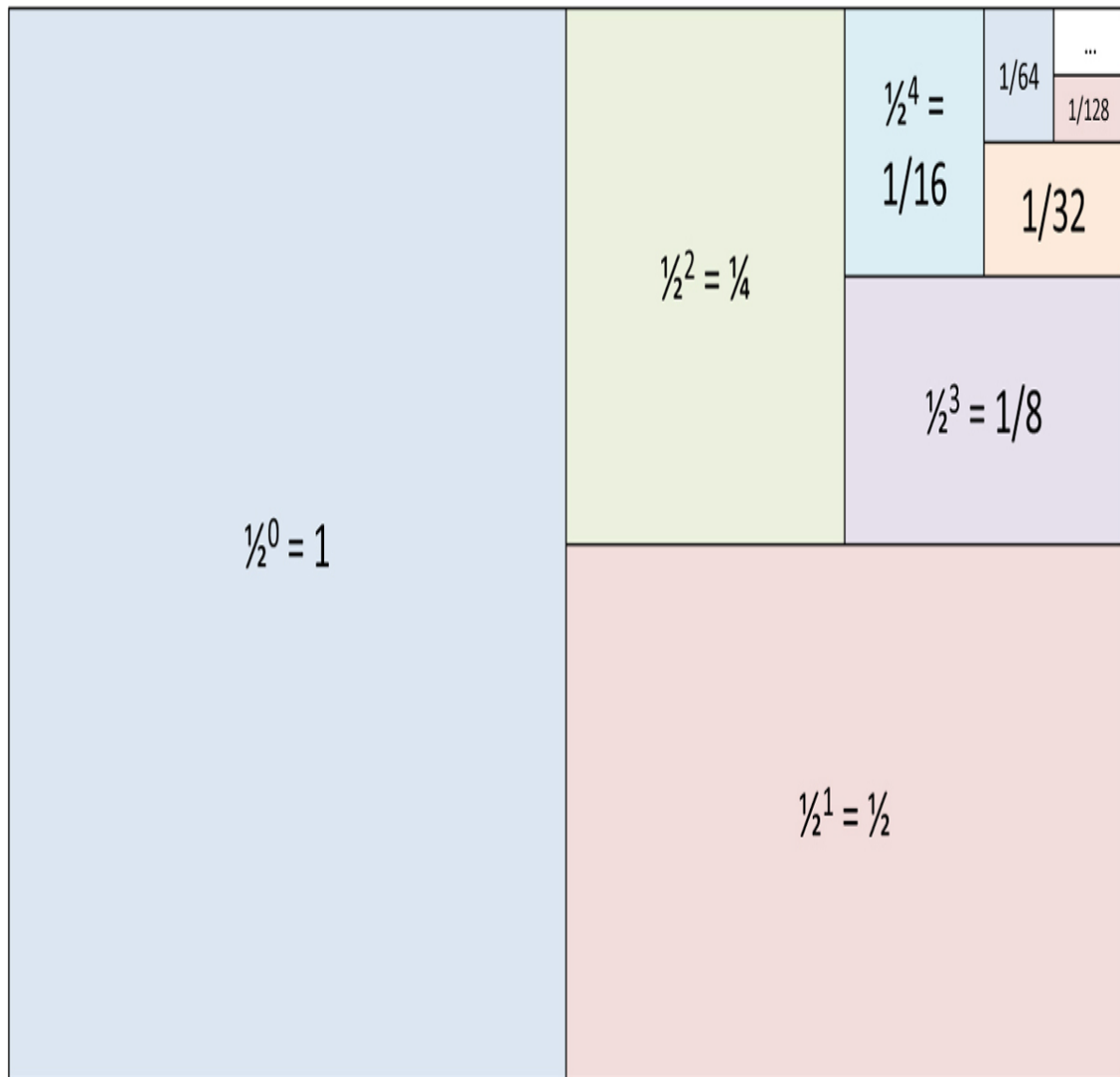
Figure 13-11 *Sifting down into two correct subheaps makes a correct heap*

Remember that the `remove()` algorithm extracts the root item, replaces it with the last item in the heap, decrements the item count, and sifts the root item down. In that case, an out-of-order item was placed in node 0, and both its subheaps were correctly ordered. After it was sifted down, the heap condition was restored. That property generalizes to work at any node in the heap tree. To be more specific, if you apply `_siftDown()` to node *J* and both *J*'s left and right subheaps satisfy the heap condition, then the subheap rooted at *J* will also satisfy the heap condition after the sift down operation completes. This property holds whether or not the item at node *J* satisfied the heap condition before sifting down.

In [Figure 13-11](#), if an out-of-order item is placed in node *J*, the subheap rooted at node *J* does not meet the heap condition. If you then sift down that item, subheap *J* will correctly satisfy the heap condition if both its child subheaps—rooted at nodes *L* and *R*—are correct. When subheap *J* is correct, you can then consider what to do with the subheap at its parent, which may not be correct.

This example suggests a way to transform an unordered array into a heap. You can apply `_siftDown()` to the nodes on the bottom of the (potential) heap—that is, at the end of the array—and work your way upward to the root at index 0. At each step, the subheaps below you will already be correct heaps because you already applied `_siftDown()` to them. When you apply it to the root, the unordered array will have been transformed into a heap.

Notice also, that the nodes on the bottom row—those with no children—are already correct heaps because they are trees with only one node; they have no relationships that can be out of order. Therefore, you don't need to apply `_siftDown()` to these nodes. You can start at node $N/2 - 1$, the rightmost node with children, instead of $N - 1$, the last node. Thus, you need only half as many sift operations as you would using `insert()` N times. [Figure 13-12](#) shows that in a 13-node heap, sifting down starts at node 5, then node 4, and so on until it reaches the root at 0.



$$\sum_{j=0}^{\infty} \frac{1}{2^j} = \frac{1}{1-\frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2$$

Figure 13-12 The `_siftDown()` method is applied to the internal nodes

In Python, the first internal node of an N node heap has index $N // 2 - 1$. So, to apply `_siftDown()` to the internal nodes, working back to the root, you could

run

```
for j in range(N // 2 - 1, -1, -1):  
    theHeap._siftDown(j);
```

Using the Same Array

The initial code fragment for the heapsort showed unordered data in a sequence. This data was then inserted into a heap and finally removed from the heap and written back to an array in sorted, descending order. In this procedure, three size- N structures are required: the initial sequence, the array used by the heap, and the result array.

In fact, the same array can be used for all three: the input, the heap, and the result array. Reusing the array cuts the amount of memory needed for heapsort by two-thirds; no memory beyond the initial array is necessary. In other words, you can create the heap *in place*. This approach only works, of course, if the source data is in an array to start. To use the heapsort algorithm on data stored in a hash table or linked list, the items would need to be inserted into a heap and then removed (or the underlying array returned as the result).

You've already seen how `_siftDown()` can be applied to half the elements of an array to transform them into a correct heap. If you transform the unordered array data into a heap in place, only one array is necessary for this task. Thus, the first step in heapsort requires only one array.

What about the second step when you apply `remove()` repeatedly to the heap? Where are you going to put the items that are removed?

Each time an item is removed from the heap, an element at the end of the heap array becomes empty because the heap shrinks by one. You can put the removed item in this newly freed cell. As more items are removed, the heap array becomes smaller and smaller, while the array of ordered data becomes larger and larger. In other words, part of the array is holding the heap, and part is holding the sorted output. With a little planning, it's possible for the ordered array and the heap array to share the same space.

The fact that all the work can be done in one array means that you don't need a separate heap object to hold the intermediate results. You can simply operate on the input array and **heapify** it. The array starts off unsorted. All N items are part of the unsorted section. When you construct the heap, the leaf nodes—the second half the array—are automatically part of the heap. The first half of the

array contains the unsorted section. As you sift items down, the unordered section of the array shrinks as the heap grows. When the array is fully heapified, then you can start removing the maximum item from index 0 and placing it at the end of the array. That way, you avoid disrupting the heap because the heap has shrunk by one.

The Heap Visualization tool can heapify an array of data items by their keys. To see an example, use the Erase and Random Fill button to erase any existing data and fill the array with a few dozen items. While the array is full, the heap tree shows only the root node, as in [Figure 13-9](#), because the random arrangement of data is unlikely to satisfy the heap condition.

Start the *heapification* by selecting the Heapify button. [Figure 13-13](#) shows a snapshot during the processing of a 30-item array. After creating the 15 leaf nodes, the algorithm steps backward through the internal nodes just above the leaves. In the figure, 6 of the internal nodes have been made into subheaps, with 5 of them satisfying the heap condition locally. The arrow labeled j indicates that it is about to sift down item 37 to make that subheap satisfy the heap condition. After it finishes making all the 3-node (and one 2-node) subheaps, it goes to the next higher level and joins them into 7-node (and one 6-node) subheaps. Note that the j arrow points at item 37 in both the array on the left and the partial tree on the right, indicating the boundary between the array cells that have been heapified so far.



Figure 13-13 *Heapifying an array of 30 random items in the Visualization tool*

With the data now in the form of a heap, the heapsort can begin producing the final, sorted ordering. The Visualization tool doesn't show the sorting process,

so let's follow the full process in a small example. [Figure 13-14](#) shows the sorting of a six-item array in place using heapsort. The first row shows the heapification process, and the second row shows the construction of the sorted array.



Figure 13-14 *Heapsort in place*

The array in the upper left of the figure is the original, unsorted array. The next step to the right shows the starting state in heapifying the array. The last three items of the array—indices 3 through 5—are treated as leaf nodes in the partial heap (shown by the dashed line encircling those nodes). You could also call these leaves “single node subheaps.” Note that not one of the items in the array has changed; a range of those nodes/cells has simply been marked as the heap area.

The third step shows what happens after calling `siftDown()` on the node at index 2. The original item in node 2 has key 28. That gets sifted down to node 5 and item 75 moves up. The heap now has four items—indices 2 through 5. The figure shows the other items in nodes 0 and 1, but they are not part of the heap. (The Visualization tool shows a node at the root of the heap tree during heapification, but it is not connected to the other nodes and is not part of the collection of verified subheaps.)

The fourth step shows the outcome of sifting down item 36 from node 1. It swaps places with item 89 in node 4, and the heap now has five items. The red links show that the heap condition has been confirmed for the connected nodes.

In the fifth step, the final item at node 0, item 51, is sifted down. It swaps places with item 89 but doesn't need to sift down any further to the second

level. Now that the six-item array is fully heapified; it's a complete, correct heap. That is shown at the right end of the top row of [Figure 13-14](#).

The heapsort algorithm then removes items from the top of the heap and places them at the end of the array. Removing the maximum item to place at the end of the array effectively swaps the root item with last item in the heap. In the example in the figure, item 89 at the root is swapped with item 28 at node 5. Then item 28 sifts down into node 2 as shown in the sixth step (first step of the second row) of [Figure 13-14](#). Item 89 sits at the end of the array, which is outside of the heap range of 0 through 4.

The seventh step takes the maximum item at the root, item 75, swaps it with the last item of the heap, item 36, and sifts item 36 down. This time the item goes down the left branch to node 1. Item 51 moves up to the root. The extracted maximum, 75, ends up in node 4. Now the four-item heap spans indices 0 through 3, and indices 4 and 5 hold the partial result.

The eighth and ninth steps are similar. They swap the root and last items of the heap, and then sift down from the root. Each time, the maximum item from the root is placed just before the other sorted items from previous steps. The heap shrinks and ends up with just one element in the heap. At this point, the array is fully sorted. The one-item heap contains the item with the minimum key. There's no need to swap it with the last heap item, itself, and do any sifting.

Going through the details shows the very efficient use of the array. The heap starts as the second half of the array, and heapify grows it to include all the items up to the root. Then the heap shrinks while occupying the first part of the array as items are removed. Let's look at the code to do this.

The `heapsort()` Subroutine

Here, we combine the two techniques—converting the array into a heap (“heapifying” it) and then placing removed items at the end of the same array—together in a subroutine that performs heapsort on an input array.

The `heapsort()` subroutine shown in [Listing 13-7](#) takes a Python array (`list`) and sorts its items by their keys as extracted by the `key` function, which defaults to the `identity()` function. The first statement sets a local variable, `heapHi`, to manage the range of indices where the heap is stored. We use the same mapping of array indices to binary tree nodes as in the `Heap` class. It calls the

heapify() routine to organize the items into a heap. Let's first discuss heapify() and then return to heapsort().

Listing 13-7 *The heapsort(), heapify(), and siftDown() Subroutines*

```
def heapsort(array,          # Sort an array in-place by keys extracted
              key=identity): # from each item using the key function
    heapHi = len(array)      # Make entire array from 0 to heapHi
    heapify(array, heapHi, key) # into a heap using heapify
    while heapHi > 1:        # While heap has more than 1 item
        heapHi -= 1          # Decrement heap's higher boundary & swap
        array[0], array[heapHi] = array[heapHi], array[0] # max and last
        siftDown(array, 0, heapHi, key) # & sift down item moved to top

def heapify(array,          # Organize an array of N items to satisfy
              N=None,       # the heap condition using keys extracted
              key=identity): # from the items by the key function
    if N is None:           # If N is not supplied,
        N = len(array)      # then use number of items in array
    heapLo = N // 2         # The heap lies in the range [heapLo, N)
    while heapLo > 0:       # Heapify until the entire array is a heap
        heapLo -= 1         # Decrement heap's lower boundary
        siftDown(array, heapLo, N, key) # Sift down item at heapLo

def siftDown(array,        # Sift item down in heap starting from
              j,           # node j
              N=None,      # down to but not including node N
              key=identity): # using key function to extract item's key
    if N is None:          # If N is not supplied,
        N = len(array)     # then use number of items in array
    firstleaf = N // 2     # Get index of first leaf in heap
    if j >= firstleaf:     # If item j is at or below leaf level,
        return             # it cannot be moved down
    item = array[j]        # Store item at cell j
    itemkey = key(item)     # and its key
    while j < firstleaf:   # While j above leaf level, find children
        left, right = j + j + 1, j + j + 2 # Get indices of children
        maxi = left        # Assume left child has larger key
        if (right < N and   # If both children are present, and
            key(array[left]) < # left child has smaller
            key(array[right])): # key
            maxi = right    # then use right child
        if (itemkey <       # If item j's key is less
            key(array[maxi])): # than max child's key,
            array[j] = array[maxi] # then move max child up
            j = maxi         # and continue from new "hole"
```

```
        else:                                # If item j's key is greater than or equal
            break                             # to larger child, then found position

array[j] = item                             # Move item to its final position
```

The `heapify()` subroutine determines the amount of the data within the array, `N`, in case the caller did not provide it. The lower bound of the heap, `heapLo`, is set to `N // 2` because all the leaf nodes are correct, single-item subheaps. Thus, the heap covers the cells from `heapLo` to `N - 1`. The `while` loop expands the heap by reducing `heapLo` to incorporate all the nonleaf nodes. It starts with the node immediately before the heap, at `heapLo - 1`. That's the rightmost internal node at the lowest level of the tree. The item there is sifted down by calling the `siftDown()` routine on the array and passing the heap bounds, `heapLo` to `N`. This loop continues until `heapLo` reaches 0, which means all the internal nodes have been sifted into the heap.

When `heapify()` is done, control returns to `heapsort()`. The `while` loop there removes items from the heap in descending order by key. Like the way the `Heap.remove()` method in [Listing 13-5](#) operates, the number of items in the heap is reduced by decrementing `heapHi`. Instead of copying the maximum item to a temporary variable, however, it is swapped with the last item of the heap. That puts the maximum item at the end of the array, in its final position for the result. The swap also moves the last leaf item to the root, and the next call to the `siftDown()` routine sifts it down into the remaining heap.

The loop stops when `heapHi` is 1 because a single-item heap needs no sifting, and the remaining item must have the smallest key. When the loop finishes, the array's items are sorted in order of increasing key values.

The `siftDown()` routine is nearly the same as the private `Heap._siftDown()` method in [Listing 13-5](#). The difference is that in the `heapsort()` context, it must pass the array and the range of indices where the heap lies. That changes as `heapsort()` runs. The operation only needs to know the index of the node to sift down, where the heap ends, and the `key` function because it does not move up in the heap tree.

If the end of the heap is not provided, the length of the array is used. The index of the first leaf node in the heap is calculated to determine when sifting down terminates. If the item to be sifted down is at or below the first leaf (in other words, has an index at or above `firstleaf`), nothing needs to be done, and the routine returns to the caller.

When node `j` is an internal node, `siftDown()` stores the item and its key in temporary variables. Then it starts a loop to perform the item moves needed to position the `item` in the correct position within its subheaps. It calculates the child node indices and determines the one with the maximum key. Then the `itemkey` is compared with the maximum child key. If the `itemkey` is smaller, the maximum child is moved up and the loop continues downward until a leaf node is reached. At the end of the loop, the `item` is moved into the cell last visited in the loop.

The Efficiency of Heapsort

As we noted, heapsort runs in $O(N \times \log N)$ time. Although it may be slightly slower than quicksort, an advantage over quicksort is that it is less sensitive to the initial distribution of data. Certain arrangements of key values can reduce quicksort to slow $O(N^2)$ time, whereas heapsort runs in $O(N \times \log N)$ time no matter how the data is distributed. Both sort methods take $O(N)$ memory, and both can sort arrays in place.

What might be somewhat surprising is that the first part of heapsort, the heapification of the array, takes only $O(N)$ time. Extracting the full sorted sequence is what requires the $O(N \times \log N)$ time. You can take advantage of that capability for other algorithms such as in statistics.

Order Statistics

Heaps have another special application: calculating **order statistics**. When you analyze large quantities of data, a variety of statistical measures provide information about the overall distribution of data. When the data can be ordered—from smallest to largest, darkest to brightest, earliest to latest, and so on—you typically want to know the minimum and maximum values. They are easy to compute by going through all the values once and updating variables that store the minimum and maximum values found.

Word clouds are a common example of the use of order statistics. In word (or text) clouds, the frequency of words in a collection such as books, chat messages, essays, or speeches is used to determine the size of the words in a graphic. [Figure 13-15](#) shows a word cloud derived from several of Mahatma Gandhi's speeches. The largest words were the most common, and the word size indicates how frequently the other terms appeared. (Very common words

such as *is*, *be*, *a*, and *to* are not shown. The speech transcripts are from www.mkgandhi.org/speeches/speechMain.htm, and the word cloud was produced by www.wordclouds.com.)



Figure 13-15 *Word cloud made from several of Mahatma Gandhi's speeches*

What if you want to find the median value? The median value lies above half the data observations and below the other half. The median is a very useful statistic because it is less sensitive to changes at the ends of the distribution. Other analyses look for the quartiles or deciles. That is the value in which a quarter or a tenth of the data is less than the value. These values can be used, for instance, to identify the least used or most used routers in a network, or the largest contributors to a political campaign.

To find these order statistics like the decile, quartile, or median, and perhaps go a step further to not just identify the value but also collect all the records that fall within, say, the highest decile, you could sort the data. With heapsort or quicksort, you can do that in $O(N \times \log N)$ time. That's fast, but still quite a bit more than the $O(N)$ time it takes to find the minimum, maximum, and linear statistics like the average. Is there a faster way?

Partial Ordering Assists in Finding the Extreme Values

As you've seen, the maximum (or minimum) keyed item goes to the root of a heap. If you want to find the 10 highest keyed items, you can simply remove 10 items from the heap. Removing these items could save quite a bit of work

compared with using heapsort to fully sort, say, 10 million records, just to find the 10 highest. So, you need to put the data in the heap efficiently and remove only the desired items.

How much time does it take to make the heap? As you saw in heapsort, if the data is already in an array that can be modified, you can heapify the items by sifting down all the items stored in internal nodes. As shown in [Listing 13-7](#), the `heapify()` subroutine consists of a single loop over the data calling `siftDown()` on each of the internal nodes.

Getting the highest keyed items from the array operates similarly to `heapsort()`, but removes only a fixed number, *K*, of items. The `highest()` function shown in [Listing 13-8](#) starts off by determining the number of items in the array and then heapifying it. A `result` array of *K*-elements is allocated to hold the maximum-keyed items.

Listing 13-8 *Subroutine to Get the K Highest Keyed Items in an Array*

```
def highest(K, array,          # Get the highest K items from an array
            N=None,           # of N items by heapifying the array based
            key=identity):    # on keys extracted by the key function
    if N is None:              # If N is not supplied,
        N = len(array)         # then use number of items in array
    heapify(array, N, key)      # Organize items into a heap
    result = [None] * K        # Construct an output array
    heapHi = N                  # End of heap starts at last item
    while N - heapHi < K:      # While we have not yet removed K items,
        result[N - heapHi] = array[0] # Put max from heap in result
        heapHi -= 1            # Decrement heap's higher boundary & swap
        array[0], array[heapHi] = array[heapHi], array[0] # max and last
        siftDown(array, 0, heapHi, key) # & sift down item moved to top

    return result              # Return K-item result
```

The `highest()` function uses a `while` loop to remove the *K* items from the heap. Each removal shrinks the heap, lowering the `heapHi` index by one, and the count of items removed is the difference between the end of the array, *N*, and `heapHi`. That difference is used as the index in the `result` array for storing the output items (in descending order by key). The `heapHi` index is then decremented to effectively remove the maximum-keyed item from the heap. The first and last items on the heap are swapped, and the new item at the root is sifted down into the correct heap position.

When the `highest()` function returns, the result array contains the K highest-keyed items in descending order, and the input array has been rearranged to satisfy the heap condition in the first $N - K$ cells, followed by K cells of the highest-keyed items in increasing order by key.

The Efficiency of K Highest

How much work is done in computing `highest()`? There are two phases to consider. The first phase is the `heapify()` operation on N items, and the second is the removal of the K highest-keyed items.

Analyzing the complexity of the second phase is straightforward. Removing an item causes one item to sift down through the heap. It's possible that the item sifts down to the leaf level of the heap. The full heap has $\log_2(N+1)$ levels, so the removal phase takes $O(K \times \log N)$ time. The heap does shrink by one for each removal, but we can assume that N is much bigger than K , and K is 1 or larger, so $\log(N)$ doesn't change significantly as K items are removed.

The analysis of the first phase might be a surprise. As mentioned earlier, the `heapify()` operation takes $O(N)$ time, even though it must go through half of the items in the array, sifting them down into the heap. That means the overall complexity of computing `highest()` is $O(N + K \times \log N)$. When K is much smaller than N , that is quite an improvement over a heapsort that takes $O(N \times \log N)$.

To understand how `heapify()` takes $O(N)$ time, let's first imagine that the sift down operation does not depend on the depth of the heap. Then each internal node would take $O(1)$ time, so processing all $N/2$ of them would take $O(N)$ time. You can think of that as a lower bound on how fast `heapify()` computes.

The next step is to look at how much work is done for each internal node. All the leaf nodes and perhaps some internal nodes have been processed to form a group of subheaps, as shown in [Figure 13-16](#). The dashed line surrounds the nodes that have been processed so far, and the red circle surrounds the next item to sift down.



Figure 13-16 *Sifting down during heapify*

The item sifting down lies at a particular level, L , of the overall heap. It can sift down all the way to the leaf level, which lies at $\log_2(N+1)$. The figure shows the difference between L and the leaf level as the *sift depth*. For the leaves, the sift depth is zero. For the parents of the leaves, it's one. Each level higher in the tree adds one until you reach the root, where the sift depth is $\log_2(N+1)$.

The sift depth limits the amount of work that must be done for a particular node. If the sift depth is S , it will take at most $S \times 2$ comparisons and S copies to shift the stored items along the path to the leaf nodes. The reason is that you need one comparison to find the maximum child, one comparison of the maximum child with its parent, and one copy to move the maximum child up (in the worst case).

Now you have a maximum bound on the work at each node. Let's see what the whole tree could require. Assume that you have a complete binary tree where every level is full of nodes. Down at the leaf nodes, the sift depth is zero, so you don't even try to sift them down. Half of the tree's nodes are leaf nodes (technically, $(N + 1) / 2$). They don't add anything to the total work.

One level up from the leaves, you have $S = 1$, a sift depth of one. How many nodes are at that level? Exactly half as many as there are leaves, so $(N + 1) / 4$, need to sift down at most one level. You can start writing down the total work done on these first two levels by multiplying the sift depth with the number of nodes at that depth:

$$\text{total work in first two levels} = 0 \frac{N + 1}{2} + 1 \frac{N + 1}{4}$$

In this case, 1 is the amount of work when $S = 1$. If you count comparisons and copies equally, then it might actually take 3 units of work. Because that 3 is a constant, and constants are ignored later when looking at the Big O complexity, you leave that constant out.

Moving up a level, you reach $S = 2$ and the number of nodes is halved again:

$$\text{total work in first three levels} = 0 \frac{N+1}{2} + 1 \frac{N+1}{4} + 2 \frac{N+1}{8}$$

Let's move the common $(N+1)$ part out of the sum for all the levels:

$$\text{total work in first three levels} = (N+1) \left(0 \frac{1}{2} + 1 \frac{1}{4} + 2 \frac{1}{8} \right)$$

As you go up in sift depth, a pattern emerges. Each term in the sum multiplies the sift depth, S , with a fraction that is a power of $\frac{1}{2}$. When you sum up K of those levels, you get

$$\text{total work in first } K \text{ levels} = (N+1) \left(0 \frac{1}{2} + 1 \frac{1}{4} + 2 \frac{1}{8} + \dots + (K-1) \left(\frac{1}{2} \right)^K \right)$$

Notice that most of the internal nodes lie near the bottom of the tree and have a small sift depth (half of them have sift depth 0). The root of the tree has the largest sift depth, but there's only one root node. So, you must process many nodes with short distances to sift down, and fewer nodes with longer distances. In the summation, each term that's added is smaller than the term before it. That means that it might be going down so fast that it doesn't keep expanding the total for more levels.

Let's now write the summation compactly. You sum from sift depth 0 up to the maximum sift depth at the root, $\log_2(N+1)$. You use S to stand for the sift depth. That makes the total work for all levels:

$$\text{total work} = (N+1) \sum_{S=0}^{\log_2(N+1)} S \left(\frac{1}{2} \right)^{S+1}$$

The first term in the total work is $(N+1)$. If that fancy summation on the right doesn't grow with N , then you end up with the work taking $O(N)$ time! Is that

even possible?

To answer that, you need to bring in some math knowledge of other summations.

$$\sum_{j=0}^{\infty} x^j = \frac{1}{1-x} \quad \text{when } -1 < x < 1$$

$$\sum_{j=0}^{\infty} jx^j = \frac{x}{(1-x)^2} \quad \text{when } -1 < x < 1$$

The first summation is easy to see visually. [Figure 13-17](#) shows the example when x is $\frac{1}{2}$. When $\frac{1}{2}$ is raised to the power 0, it is one and is represented by the square on the left. When it's raised to the power 1, it is one-half that size and is represented by the rectangle on the lower right. With each successive power, you divide the last rectangle in half and place it in the space at the upper-right corner.



Figure 13-17 *Infinite sum of the powers of $\frac{1}{2}$*

These ever-shrinking rectangles continue to fill up *half of the remaining space*. They never shrink to zero, but the space remaining continues to shrink. As the power goes to infinity, the total space of all the rectangles sums to 2. How do you know that? If you look at the rectangle that holds all the squares and little rectangles in the figure, it's exactly twice as big as the first square, which contributed exactly 1 unit to the total. Looking at the first formula and substituting $\frac{1}{2}$ for x , you get $1 / (1 - \frac{1}{2})$, which is 2.

The second summation formula comes by taking the derivative of both sides of the first equation and then multiplying both by x . That's hard to see visually. If you haven't seen it before, it's part of calculus.

You can use the second summation to simplify the total work equation. By summing all the way to infinity, instead of just to $\log_2(N+1)$, you get an upper bound on the work done by `heapify()`.

$$total\ work = (N + 1) \sum_{S=0}^{\log_2(N+1)} S \left(\frac{1}{2}\right)^{S+1} < (N + 1) \sum_{S=0}^{\infty} S \left(\frac{1}{2}\right)^{S+1}$$

The summation over S doesn't quite match the formula with jx^j but you can modify it by factoring out one of the powers of $\frac{1}{2}$:

$$total\ work < \frac{N + 1}{2} \sum_{S=0}^{\infty} S \left(\frac{1}{2}\right)^S$$

Substituting $\frac{1}{2}$ for x in the second summation formula above, you get

$$total\ work < \frac{N + 1}{2} \sum_{S=0}^{\infty} S \left(\frac{1}{2}\right)^S = \frac{N + 1}{2} \frac{\frac{1}{2}}{\left(1 - \frac{1}{2}\right)^2} = \frac{N + 1}{2} \frac{1}{\frac{1}{4}} = N + 1$$

So, the upper bound on the total work performed by `heapify()` is just a multiple of N , and that means `heapify` runs in $O(N)$ time. Proving that took somewhat complicated math but was worth the effort. It means that `highest()` runs in $O(N + K \times \log N)$ time to find the K highest keyed items. Compared with a `heapsort()` that takes $O(N \times \log N)$ time, it saves $O((N - K) \times \log N - N)$ time. The bigger the difference between N and K , the more important it is to use `highest()`.

Summary

- A heap is a fundamental data structure that organizes items in a partial ordering within a binary tree.
- The item with the largest key is stored at the root of the heap tree.

- The heap condition requires that the key of an item at a node of the tree is greater than or equal to those of its child nodes, if any.
- The binary tree for the heap is always complete. All levels of the tree are completely full, and the lowest level can only be missing nodes on the right.
- Priority queues are typically constructed using heaps either as descending-priority queues or ascending-priority queues (where the smallest keyed item is at the root of the heap tree and all children have keys greater than or equal to their parent's).
- Heaps and priority queues offer common methods: inserting a new item, removing the largest item, and peeking at the first (largest) item.
- A heap offers removal of the largest item, and insertion, in $O(\log N)$ time.
- Heaps support traversal but do *not* support *ordered* traversal of the data, locating an item with a specific key, nor deletion.
- Heaps are usually implemented as arrays, taking advantage of the completeness of the binary tree to uniquely map array cells to tree nodes.
- The root is at index 0 and the last item at index $N-1$.
- The completeness of the tree ensures that all cells between 0 and $N-1$ are used.
- Insertion operates by placing an item in the first vacant cell of the array and then sifting it up to its appropriate position.
- When an item is removed from the root, it's replaced by the last item in the array, which is then sifted down to its appropriate position.
- The sift up and sift down processes can be thought of as a sequence of swaps but are more efficiently implemented as a sequence of copies.
- If the heap supports changing the priority of an arbitrary item, it must first locate the node with the item. Next, its key is changed. Then, if the key was increased, the item is sifted up, and if the key was decreased, the item is sifted down.

- A heap can be implemented using nodes and references to form the binary tree (not a search tree). This is called a tree-based heap.
- Algorithms exist to find the last occupied node or the first free node in a tree-based heap using the number of items in the tree.
- Heapsort is an efficient sorting procedure that requires $O(N \times \log N)$ time.
- Conceptually, heapsort consists of making N insertions into a heap, followed by N removals.
- Heapsort can be made to run faster by applying the sift down algorithm directly to $N/2$ items in the unsorted array, rather than inserting N items.
- The same array can be used for the initial unordered data, for the heap array, and for the final sorted data. Thus, heapsort requires no extra memory.
- The heapify operation organizes the items in an array to satisfy the heap condition.
- Heapifying an array takes $O(N)$ time.
- Finding the K highest (or lowest) keys among N items in an array can be done efficiently using heapify followed by K removals.
- The complexity of finding these K order statistics is $O(N + K \times \log N)$.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. What does the term *complete* mean when applied to binary trees?
 - a. All the necessary data items have been inserted.
 - b. All the rows are filled with nodes, except possibly the bottom one.
 - c. All existing nodes contain data.
 - d. The node arrangement satisfies the heap condition.
- 2 What does the term *partially ordered* mean when applied to heaps?

3. When an item is removed from a heap, it is always removed from the _____.
4. When an item is inserted into a heap,
 - a. a hole is introduced at the root node and sifted down until it reaches the position the item should occupy.
 - b. a search starts from the root to find the item with the key equal to or just above the key to insert, and the item is inserted as that node's child.
 - c. the maximum item in the heap is moved to the first empty cell, and the new item is inserted and sifted down until it reaches the position it should occupy.
 - d. the item is inserted in the first empty cell and then sifted up until it reaches the position it should occupy.
5. A heap can be represented by an array because a heap
 - a. is a binary tree.
 - b. is partially complete.
 - c. is partially ordered.
 - d. satisfies the heap condition.
6. The last node in a heap is
 - a. always a left child.
 - b. always a right child.
 - c. always on the bottom row.
 - d. never less than its sibling.
7. A heap is to a priority queue as a(n) _____ is to a stack.
8. Which operation is more complex, sifting up or down? Why?
9. The basic heapsort concept involves
 - a. removing data items from a heap and then inserting them again.
 - b. inserting data items into a heap and then removing them.
 - c. copying data from two heaps into another empty one, merging their items.

- d. copying data from the array representing a heap to the heap tree.
- 10. How many arrays, each big enough to hold all the data, does it take to perform a heapsort?
- 11. The time complexity of running heapsort is $O(\text{_____})$.
- 12. Compared to quicksort, heapsort is _____ .
- 13. To heapify an array:
 - a. the items in the second half of the array are sifted up.
 - b. the items in the array are sequentially inserted into a separate heap data structure.
 - c. the array is reordered in place to establish the heap condition between nodes.
 - d. the heapsort algorithm is applied to the leaf nodes of the partial heap tree.
- 14. The `heapify()` routine on an N-item array takes $O(\text{_____})$ time.
- 15. To get the K highest keyed items from an N-item array takes $O(\text{_____})$ time.

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

13-A Imagine you have a heap, H , and you insert an item M on it that has a key larger than any item currently in the heap. If you now remove an item from the heap, you will get item M as the item returned. Will the heap after the two operations be identical to what it was before? By identical, we mean all the items are in the same nodes of the binary tree as they were before. Why or why not?

13-B Sorting algorithms are called stable when items with equal keys remain in the same relative order after an array of items is sorted. Does the `heapsort()` of [Listing 13-7](#) implement a stable sort? Why or why not?

13-C While the Visualization tool does not perform a heapsort, it can shed light on Experiment 13-B. Insert some items with equal keys. Then

remove them. The color of the nodes is the secondary data item. Carefully note the colors assigned to each item. Can you find examples where the items are removed in something other than the reverse of the order they were inserted?

13-D Does the order in which data is inserted in a heap affect the arrangement of the heap? Use the Heap Visualization tool to find out. Try taking a group of seven distinct keys and inserting them in different orders.

13-E Use the Visualization tool's Insert button to insert 10 items in ascending order into an empty heap. If you remove these items with the Remove Max button, will they come off in the reverse order? Does the answer change depending on which 10 items are inserted?

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

- 13.1 Add two `levels()` methods to the `Heap` class that return the number of levels in the heap. An empty heap has zero levels. A heap with one item has one level. The number of levels is one more than the level of the deepest leaf node. Implement `levels_loop()` by making a loop that descends the heap tree to find the deepest leaf, counting the levels along the way. Implement `levels()` by using Python's `math` package and its `log2()` function on the number of items in the heap. Test your program on heaps of sizes ranging from 0 to 33 items.
- 13.2 Make a version of `Heap` that accepts a flag in its constructor to select an ascending, rather than descending, heap. In other words, the item key at the root is the smallest rather than the largest, and child items have keys greater than or equal to their parent, when ascending is chosen. Make sure all operations work correctly for both ascending and descending heaps.
- 13.3 Implement a `merge()` method for the `Heap` class that takes a second heap and merges its items into the object. The method should check that the key functions are identical for two heaps and raise an exception if

they differ. Is there a faster way to merge the heaps than removing items from the second one and inserting them in the first? Hint: Think about the analysis for heapsort and K highest. Demonstrate your program merging both empty and nonempty heaps, and heaps with different key functions.

- 13.4 Implement a `replaceItem()` method for the `Heap` class that behaves like the `change_priority()` method described in the “[Other Operations](#)” section. The `replaceItem()` method should take two arguments: an item already in the heap and a new item to replace it. Because the heap only has a function to extract keys from items, it cannot update the key of the existing item, so a new item must replace it. Your implementation must find the existing item in the heap and raise an exception if it is not found. After the item is found, it should replace the item and then sift it either up or down, depending on whether the new item key is larger or smaller than the old one.

The search for the item to replace can be made more efficient by not searching subheaps where the key of the existing item exceeds that of the top of the subheap. All the keys in the subheap must have equal or lower values. This filtering of subheaps can be done by starting with a queue containing just the root index. At each iteration, remove an index from the queue. If it indexes the goal item, stop and return the value. If not, put the child node indices in the queue if their keys equal or exceed the goal key. If the queue becomes empty, the item is not in the heap.

Demonstrate the results of your program for these cases:

- Item exists in heap; replacement item has a higher key
- Item exists in heap; replacement item has a lower key
- Item not in heap and has a key higher than all those in the heap
- Item not in heap and has a key lower than all those in the heap

- 13.5 Use the word frequency-counting program that was built for Programming Project 11.5 to analyze the 20 most frequently used and 20 least frequently used words in a text file. As a reminder, the word frequency program reads a text file, extracts the individual words, and counts the number of times they occur using a hash table. The program should traverse the items in the hash table and insert them into a plain array. The array can then be heapified first in descending order and then

ascending order using the word count as the key. Use something like the `highest()` subroutine to extract and print the top 20 for each order. Be careful if the text file has fewer than 40 distinct words in it. Finding the top K words won't produce a good word cloud because the most frequently occurring words are likely to be the most common words in the source language of the text, but it still can yield interesting results.

14. Graphs

In This Chapter

- [Introduction to Graphs](#)
- [Traversal and Search](#)
- [Minimum Spanning Trees](#)
- [Topological Sorting](#)
- [Connectivity in Directed Graphs](#)

Graphs are among the most versatile structures used in computer programming. They appear in all kinds of problems that are generally quite different from those we've dealt with thus far in this book. If you're dealing with general kinds of data storage problems such as records in a database, you probably don't need a graph, but for some problems—and they tend to be interesting ones—a graph is indispensable.

Our discussion of graphs is divided into two chapters. In this chapter we cover the algorithms associated with unweighted graphs, show some problems that these graphs can represent, and present a visualization tool to explore them. In the next chapter we look at the more complicated algorithms associated with weighted graphs.

Introduction to Graphs

Graphs are data structures rather like trees. In a mathematical sense, a tree is a particular kind of graph. In computer programming, however, graphs are used in different ways than trees.

The data structures examined previously in this book have an architecture dictated by the algorithms used on them. For example, a binary tree is structured the way it is because that “shape” makes it easy to search for data

and insert new data. The links between nodes in a tree represent quick ways to get from parent to child.

Graphs, on the other hand, often have a shape dictated by a physical or abstract problem. For example, nodes in a graph may represent cities, whereas edges (links) may represent airline flight routes or roads or railways between the cities. Another more abstract example is a graph representing the individual tasks necessary to complete a project. In the graph, nodes may represent tasks, whereas directed edges (with an arrow at one end) indicate which task must be completed before another. In both cases, the shape of the graph arises from the specific real-world situation.

If graphs represent real-world things, what can they be used for? Well, the ones that describe transportation links can be used to find all the possible ways of getting from one place to another. If you're only interested in the shortest path (or maybe the longest), there are algorithms that use graphs to find that path. When the graph represents communication between people, you can find clusters of people that form communities or organizations. Similarly, communication graphs can be used to find people or groups that are isolated from one another.

Before going further, we must mention that, when discussing graphs, nodes are traditionally called **vertices** (the singular is **vertex**). The links between vertices are called **edges**. The reason is probably that the nomenclature for graphs is older than that for trees, having arisen in mathematics centuries ago.

Definitions

Figure 14-1 shows a simplified map of the major freeways in the vicinity of Seattle, Washington. Next to the map is a graph that models these freeways.

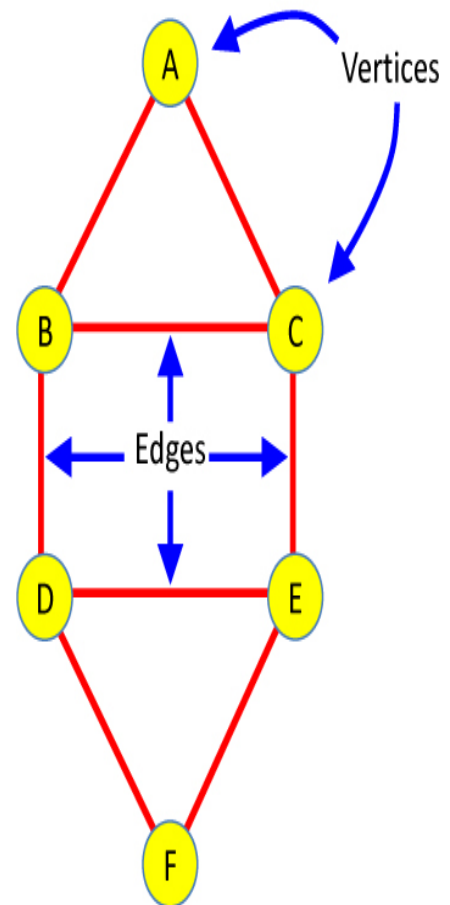
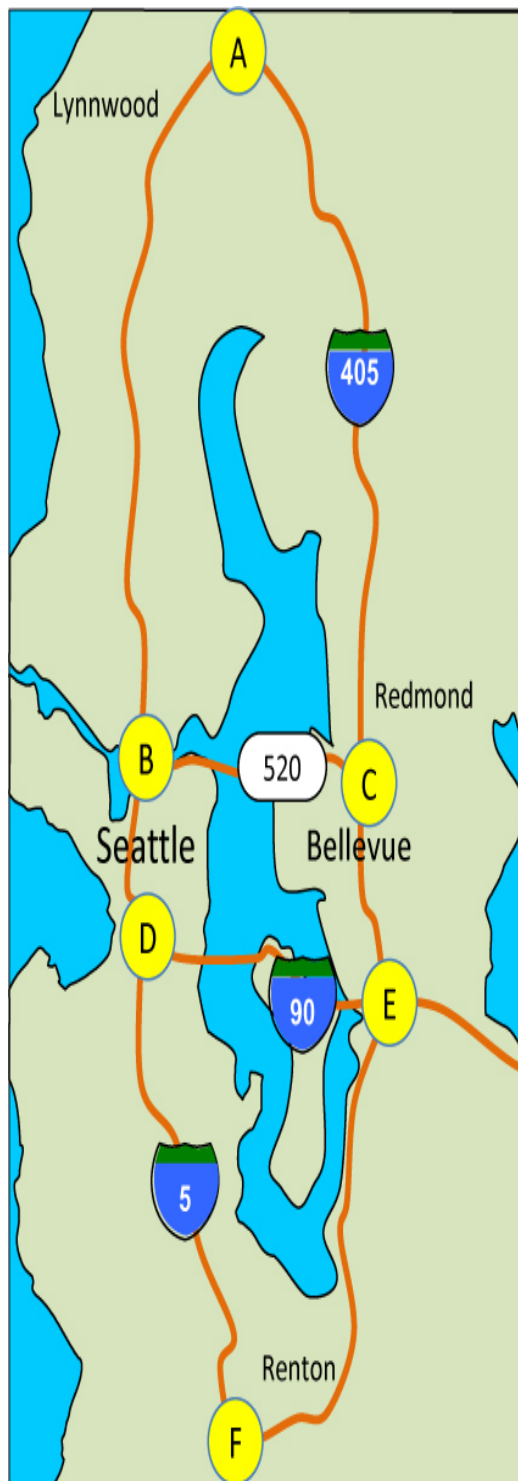


Figure 14-1 *Roadmap and a corresponding graph*

In the graph, circles represent freeway interchanges, and straight lines connecting the circles represent freeway segments. The circles are *vertices*, and the lines are *edges*. The vertices are usually labeled in some way—often, as

shown here, with letters of the alphabet. Each edge connects and is bounded by the two vertices at its ends.

The graph doesn't reflect the exact geographical positions shown on the map; it shows only the relationships of the vertices and the edges—that is, which edges are connected to which vertex. It doesn't concern itself with physical distances or directions (even though the figure shows them in roughly the same orientation as the map). The primary information provided by the graph is the *connectedness* (or lack of it) of one intersection to another, not the actual routes.

Adjacency and Neighbors

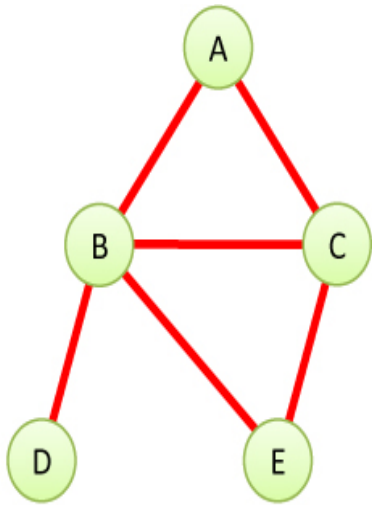
Two vertices are said to be **adjacent** to one another if they are connected by a single edge. Thus, in [Figure 14-1](#), vertices C and E are adjacent, but vertices C and F are not. The vertices adjacent to a given vertex are said to be its **neighbors**. For example, the neighbors of vertex D are B, E, and F.

Paths

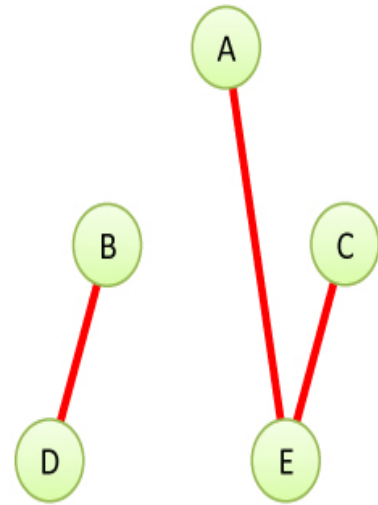
A **path** is a sequence of edges. The graph in [Figure 14-1](#) has a path from vertex A to vertex F that passes through vertices B and D. By convention, we call this path ABDF. There can be more than one path between two vertices; another path from A to F is ACEF. Because this graph came from a road network, you can easily see the correspondence of real-world routes to paths in the graph.

Connected Graphs

A graph is said to be **connected** if there is at least one path from every vertex to every other vertex, as in the graph in [Figure 14-1](#). If “You can’t get there from here” (as a rural farmer might tell city slickers who stop to ask for directions), the graph is not connected. For example, the road networks of North America are not connected to those of Japan. A **nonconnected graph** consists of several **connected components**. In [Figure 14-2](#), two graphs with the same group of vertices are shown. In the graph on the left, all five vertices are connected, forming a single connected component. The graph on the right has two connected components: B-D and A-C-E.



Connected components: 1



Connected components: 2

Figure 14-2 *Connected and nonconnected graphs*

Note that an edge always links two vertices in a graph. It would be incorrect to eliminate, for instance, vertex D in the right-hand graph of [Figure 14-2](#) and leave a “dangling” edge connected to B.

For simplicity, the algorithms we discuss in this chapter are written to apply to connected graphs or to one connected component of a nonconnected graph. If appropriate, small modifications usually enable them to work with nonconnected graphs as well.

Directed and Weighted Graphs

[Figure 14-1](#) and [Figure 14-2](#) show **undirected graphs**. That means that the edges don’t have a *direction*; you can go either way on them. Thus, you can go from vertex A to vertex B, or from vertex B to vertex A, with equal ease. Undirected graphs model rivers and roads appropriately because you can usually go either way on them (at least, slow-flowing rivers). Sometimes undirected graphs are called **bidirectional graphs**.

Graphs are often used to model situations in which you can go in only one direction along an edge—from A to B but not from B to A, as on a one-way street, the northbound or southbound lanes of a freeway, or downstream on a river with waterfalls and rapids. Such a graph is said to be **directed**. The

allowed direction is typically shown with an arrowhead at the end of the edge. A valid path in a directed graph is a sequence of edges where the end vertex of edge J is the start vertex of edge $J + 1$.

In some graphs, edges are given a numeric **weight**. The weight is used to model something such as the physical distance between two vertices, or the time it takes to get from one vertex to another, or how much it costs to travel from vertex to vertex (on airline routes, for example). Such graphs are called **weighted graphs**. We explore them in the next chapter.

In this chapter we start the discussion on simple undirected, unweighted graphs; later we explore directed, unweighted graphs. We have by no means covered all the definitions and descriptions that apply to graphs; we introduce more as we go along.

The First Uses of Graphs

One of the first mathematicians to work with graphs was Leonhard Euler in the early eighteenth century. He solved a famous problem dealing with the bridges in the town of Königsberg, on the Baltic coast. This town on a river included an island and seven bridges, as shown in [Figure 14-3](#).

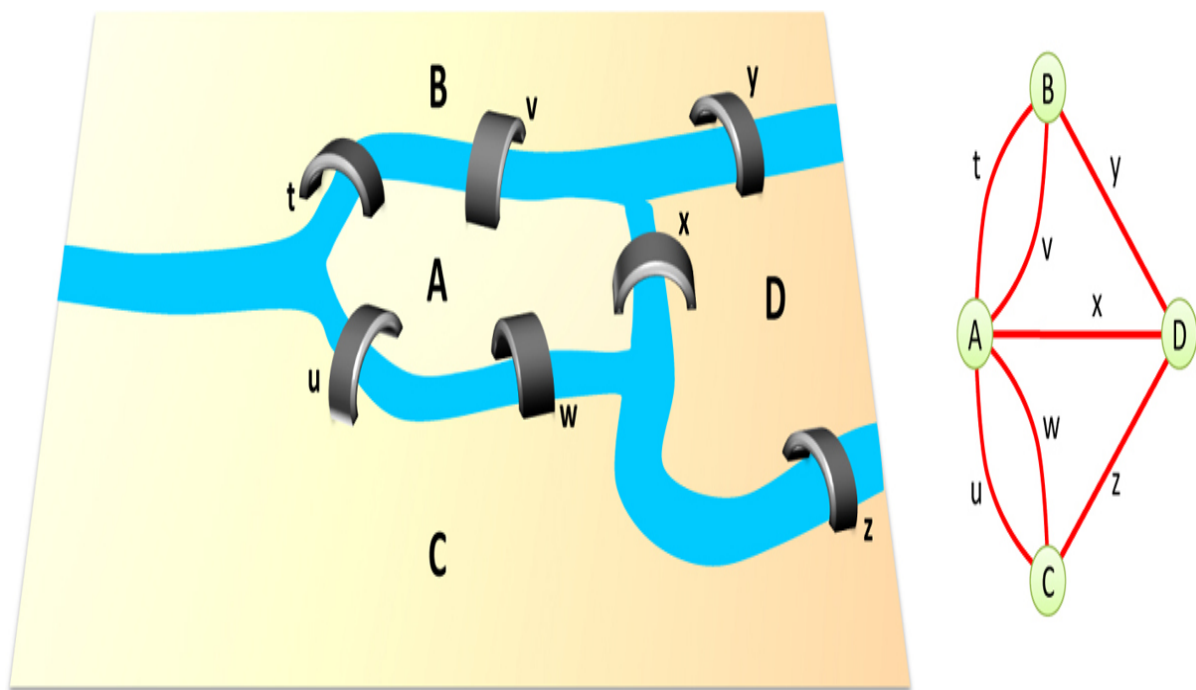


Figure 14-3 *The bridges of Königsberg*

The problem, much discussed by the townsfolk, was to find a way to walk across all seven bridges without recrossing any of them. We don't recount Euler's solution to the problem; it turns out that there is no such path. The key to his solution, however, was to represent the problem as a graph, with land areas as vertices and bridges as edges, as shown at the right of [Figure 14-3](#). This is perhaps the first example of a graph being used to represent a problem in the real world.

Note that in the graph of the Königsberg bridges, multiple bridges connect the different land areas. For example, the island, A, connects to the lower bank of the river, C, by bridges u and w. The graph shows multiple edges, and the edges are labeled to distinguish them. The term for a graph that allows multiple edges to connect a single pair of vertices is a **multigraph**. In this case, the edge labels aren't weights—just ways to distinguish the possible paths between vertices A and C.

Representing a Graph in a Program

It's all very well to think about graphs in the abstract, as Euler and other mathematicians did, but you want to represent graphs using a computer. What sort of software structures are appropriate to model graphs? Let's look at vertices first and then at edges.

Vertices

In an abstract graph program, you could simply number the vertices 0 to $N-1$ (where N is the number of vertices). You wouldn't need any sort of variable to hold the vertices because their usefulness would result from their relationships with other vertices.

In most situations, however, a vertex represents some real-world object, and the object must be described using data items. If a vertex represents a city in an airline route simulation, for example, it may need to store the name of the city, the name of the airport, its altitude, its location, runway orientations, and other such information. Thus, it's usually convenient to represent a vertex by an object of a vertex class. Our example programs store only a name string (like *A*), used as a label for identifying the vertex. [Listing 14-1](#) shows how the basic `Vertex` class might look.

Listing 14-1 *The Basic `Vertex` Class*

```
class Vertex(object):      # A vertex in a graph
    def __init__(self, name): # Constructor: stores a vertex name
        self.name = name     # Store the name
    def __str__(self):       # Summarize vertex in a string
        return '<Vertex {}>'.format(self.name)
```

Note that the name attribute is declared public here. The reason is that you can allow it to be manipulated by the caller during various operations without affecting the graph containing it.

Vertex objects can be placed in an array and referred to using their index number. The vertices might also be placed in a list or some other data structure. The unique vertex index or the object itself can identify this vertex within a graph.

For vertices with coordinates like latitude and longitude, storing them in a quadtree may make sense, as described in [Chapter 12, “Spatial Data Structures.”](#) It is important to have them in some structure that preserves their unique identifiers so that even if the caller changes the name or other attribute, the vertex can be retrieved. If you use a quadtree, you could not allow the coordinates of the vertex to change without changing its placement in the quadtree. For graphs with simply labeled vertices, an array is fine for storage if the vertex always stays at its original index.

Whatever structure is used for vertices, this storage is for convenience only. It has no relevance to how they are connected by edges. For edges, you need another mechanism.

Edges

In [Chapter 8, “Binary Trees,”](#) you saw that a computer program can represent trees in several ways. Mostly that chapter examined trees in which each node contained references to its children, but you also learned that an array could be used, with a node’s position in the array indicating its relationship to other nodes. [Chapter 13, “Heaps,”](#) described arrays used to represent a particular kind of tree called a *heap*.

A graph, however, doesn’t usually have the same kind of fixed organization as a tree. In a binary tree, each node has a maximum of two children, but each vertex in a graph may be connected to an arbitrary number of other vertices.